

AAKAAR

03-LLD

Low-Level Design (LLD)

Module-level design of the engine, governance, audit, and execution paths

Architecture

AUDIENCE	Engineers, Maintainers
VERSION	1.0
STATUS	Released
CLASSIFICATION	Confidential

Low-Level Design: The Aakaar Engine

In plain terms. This document explains, module by module, how Aakaar actually *runs the work* a user designs. Picture an automated assistant that follows a flowchart: it does one step, waits if a person needs to approve something, writes down everything it did in a tamper-proof logbook, and — crucially — can pick up exactly where it left off if the power goes out mid-task. This document is for engineers and architects who need to know *why* each part is built the way it is, not just *what* it does. It is deliberately technical, but every design choice is explained in business terms first.

Aakaar is an air-gapped automation platform for banks: a workflow is a DAG (directed acyclic graph) of capability nodes — "log into the bank portal", "download the statement", "reconcile the ledger", "ask a human for an OTP". The engine's job is to take a validated DAG and drive it to a terminal status while satisfying four hard banking requirements: **durability** (survive a crash), **governance** (maker-checker on money-moving actions), **auditability** (a tamper-evident trail), and **reversibility-awareness** (never double an irreversible side effect, and be able to *simulate* before doing it for real).

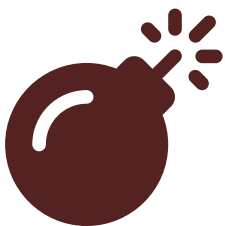
Everything runs in one Python process on top of FastAPI + SQLite (Chroma for vectors). There is no Redis, no Postgres requirement, no Temporal server, no Vault server, no S3 — that is a design constraint, not an accident. The cleverness of the engine is delivering Temporal-grade behaviour (checkpoint/resume, at-least-once event fan-out, human-in-the-loop) entirely in-process.

1. The architecture spine

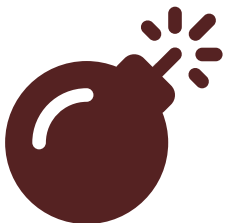
The engine is split into a *Protocol* (`Executor`) and an implementation (`LocalExecutor`). Everything above the executor — the orchestrator, repositories, the API — talks to the Protocol, never the concrete class. That single seam is what lets a bank swap in a Temporal-backed executor later without touching a single caller.

This flowchart shows how a run flows from the API down through the engine and back out to the timeline UI.

Maximum text size in diagram
exceeded



Syntax error in text
mermaid version 11.15.0



Syntax error in text
mermaid version 11.15.0